

# JEV RAG: A More Efficient Solution for RAG Systems?



Jev is a hot topic these days. Lately, about 70% of the X timeline is filled with Jev posts. TypeSafe AI announced Jev, an AI model, on September 15, 2026.

It's a classifier-like model that doesn't generate text and returns only three types of judgments: "Choice," "Score," and "Noul (Yes/No probability)."

Jev is an AI model that does not generate text, but only returns "typed judgments + calibrated probabilities." It boasts fast response times of 70–500ms and costs that are tens to hundreds of times less than LLMs.

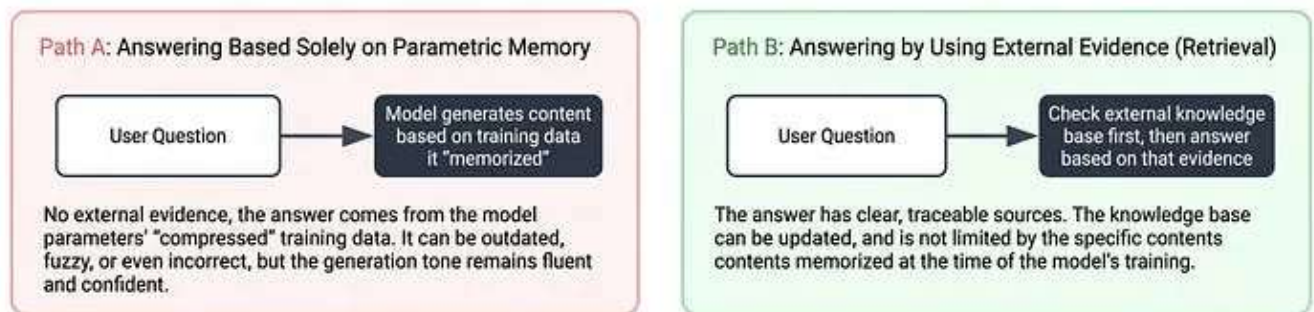
Jev's unique feature is that it "does not generate text." It's a new type of model that returns only a classification quickly and at low cost, and in publicly available validations, **its accuracy is one step below frontier LLMs.**

Nevertheless, it has value when used in a design where "Jev does classification, and Claude does generation."

The most troublesome problems in these LLM systems **Hallucination** This problem causes the model to say completely wrong things in a very confident tone, and it will keep making the same mistakes even after you keep correcting it.

But the most troublesome thing is not that the model makes mistakes. The real problem is that **the tone in which the model says the wrong thing is almost the same as the tone in which it speaks in a conversation.**

Press enter or click to view image in full size



If you ask the model for basic information like dates, errors are very likely to occur in the output. Early versions of the model also frequently produced various errors in problems requiring precise calculations, such as mathematical problems.

While current models have improved significantly in these areas, they haven't been completely solved. In addition to improvements in the model's capabilities, current models also utilise RAG (Retrieval-Augmented Generation).

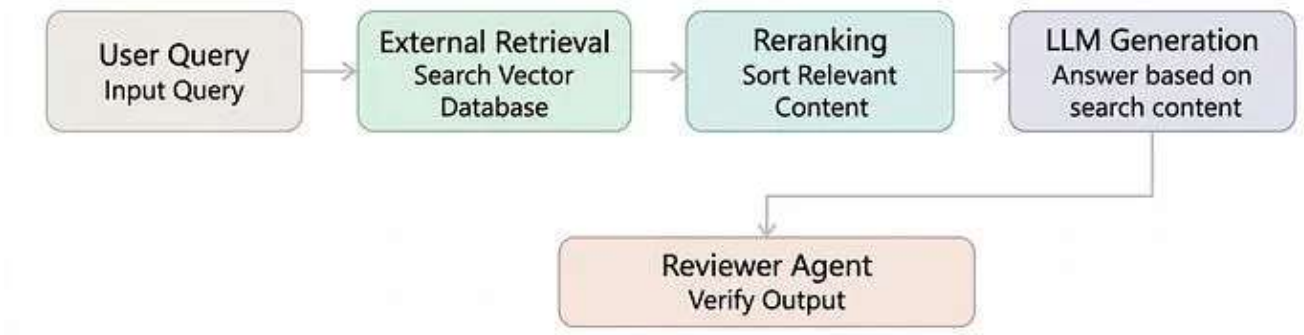
As I mentioned in many of my videos, allowing them to obtain external data when precise information is needed, further reducing the probability of generating erroneous content out of thin air.

Also, I mentioned that **knowledge shouldn't be memorised by the model, but rather retrieved when needed.**

When there's clear external evidence to trace the answer, the model can answer based on the retrieved information, rather than guessing based on impressions.

This way, at least the answer has a chance to be verified and its source traced.

Press enter or click to view image in full size



However, this illusion problem cannot be fundamentally solved at present, because even if the retrieved content is correct, the model may still misinterpret or misuse it, or make logical errors when integrating multiple search results.

This is why the validation layer in the RAG Pipeline **Reranking** and the subsequent **Reviewer Agent** ones are not superfluous designs.

Let me give you a quick demo of the live chatbot to show you how everything works.

When I open the app, it first checks the **documents/** folder. It looks at every PDF and text file and checks if anything has changed since the last time I used the app.

The first time I run the app, it reads the documents page by page. It breaks the text into small chunks. Some text overlaps between the chunks so important information is not lost.

It uses OpenAI to create **embeddings** for these chunks. The embeddings are saved in a local **Chroma vector database**. A progress bar shows you how much work is finished.

The next time you open the app, it does not need to process everything again. It loads the saved index from your computer, so it is much faster. If the app cannot read a file, such as a scanned or broken PDF, it shows the file in the sidebar instead of crashing.

In the sidebar, you can upload new files. The app automatically adds them to the index. You can also change the search settings, such as how many chunks to search, how many good chunks to keep, and how strict the relevance score should be.

I asked the question: "What is reranking?" The app shows the search process step by step. First, it changes your question into a clear search query.



Next, the app finds the 15 closest chunks from the database. A small OpenAI model checks all 15 chunks and gives each one a **relevance score from 0 to 4**.

Chunks with low scores are removed. The best chunks are sent to the main answer model.

The answer appears on the screen word by word. The AI must answer using only the information found in the documents. It also adds citations such as [1] and [2].

Below the answer, there is a **Sources** section. It shows the file name and page number for each source. It also shows the relevance score, confidence, and similarity.

Sources used in the final answer are highlighted. Sources that were found but not used are shown with less emphasis.

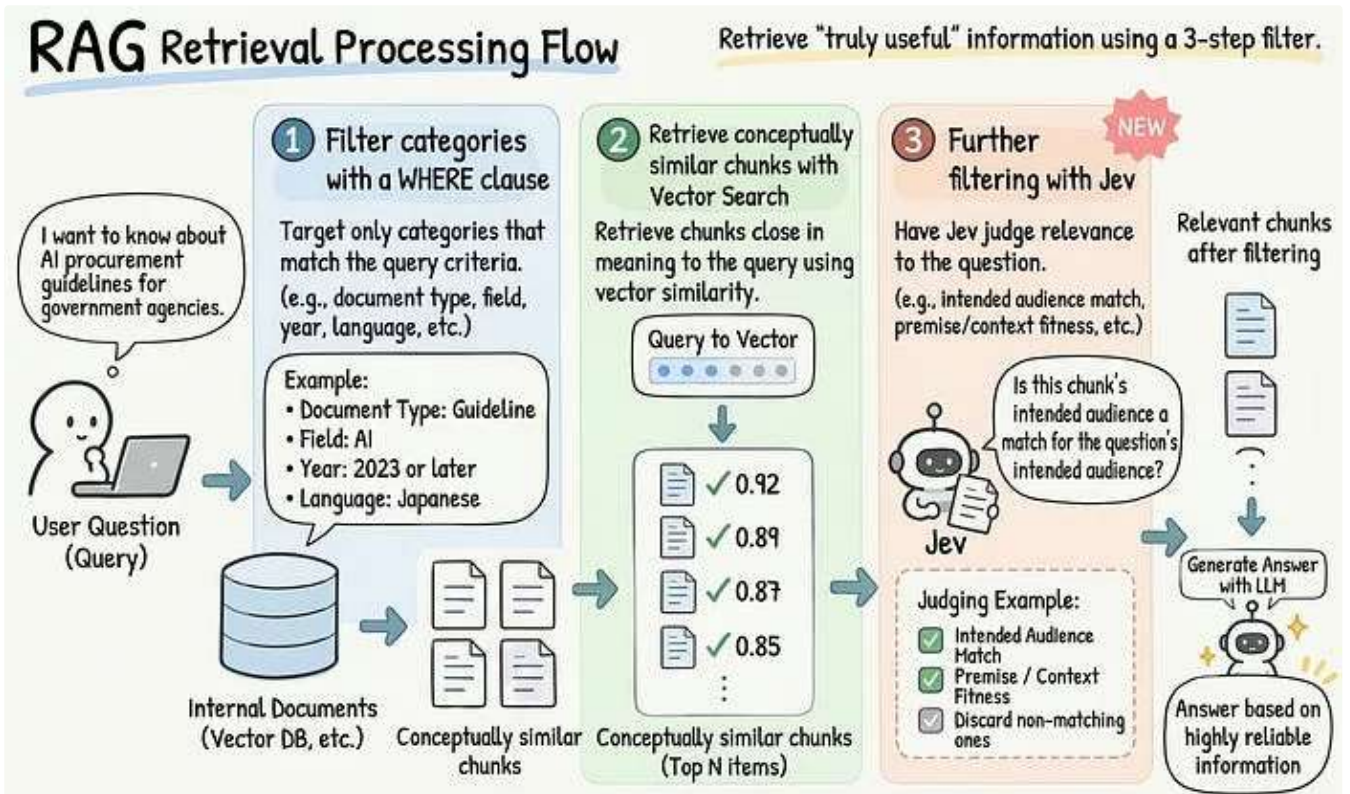
## Where to place Jev in RAG

There are two possible locations for placing Jev: before and after the vector search. Pattern 1 is to use Jev to filter out the search results, while Pattern 2 is to have Jev choose “which tags to use to narrow down the results for this question” before the search.

### Pattern 1: After searching, play with Jev

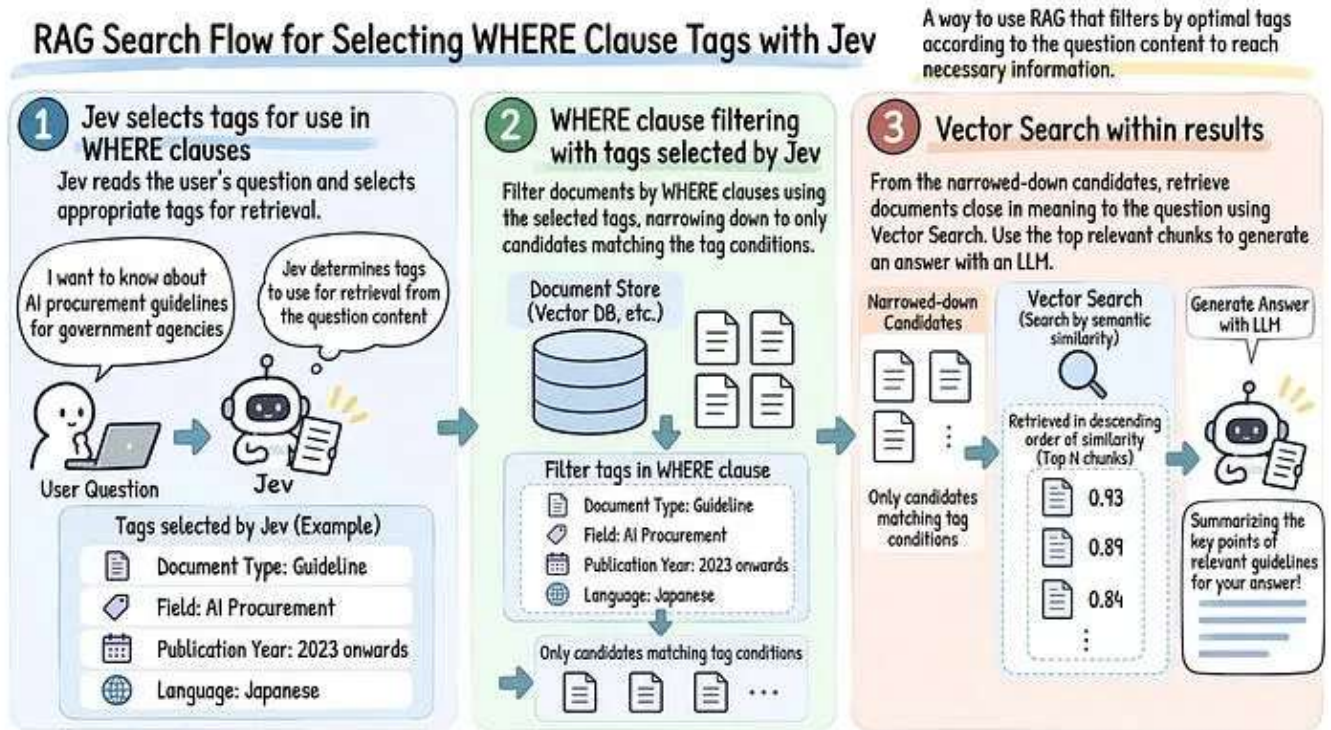
There are two possible locations for placing Jev: before and after the vector search. Pattern 1 is to use Jev to filter out the search results, while Pattern 2 is to have Jev choose “which tags to use to narrow down the results for this question” before the search.

Press enter or click to view image in full size



After retrieving candidates using a WHERE clause and vector search, Jev reads the text of the chunk to determine if the target audience is correct. Its strength lies in its ability to make judgments that can only be made by reading the text, such as when the topic is similar, but the target audience is different.

## Pattern 2: Select tags in Jev before searching.



I haven't completed a proof-of-concept (PoC) for this yet, but it seems promising, so I'll introduce it. It assumes that each record in Cosmos DB has tags (document type, field, publication year, language, etc.). A user's question is passed to Jev, who selects the appropriate tags, and then a WHERE clause is created using those tags before performing a vector search.

If the conditions are known, you can save RUs by narrowing down the results in the WHERE clause beforehand. Pattern ② is when Jev selects the conditions from the query text instead of a person providing them.

There are two ways to choose a tag.

- For items that are determined by a single factor, such as document type, the user selects one option from the choices provided in Choice. Choice can have up to 255 options, and an option for "None of the above" **other** can also be added.
- For items that may have multiple tags, a Noul request is sent for each tag. Since questions are evaluated in parallel and grouped into a single request, increasing the number of tags will not significantly change the latency.

I couldn't confirm the maximum number of questions that can be submitted at once. However, the total request is limited to approximately 64,000 tokens, so the longer you write your tag options, the larger the input will become.

One thing to be careful about is the impact of selecting the wrong tag. If you select the wrong tag, the correct chunk may not even be included in the search results.

**confidence** When Choice is low, or Noul is around 0.5, it's a good idea to have a fallback that searches the entire dataset without narrowing the search.

## What did we verify?

### Verification 1: Re-ranking

Cohere Rerank 3.5 / Amazon Rerank 1.0: Pass a question and 20 chunks to Bedrock's reranking function and sort them.

Claude Haiku 4.5: I give him 20 chunks, numbering them, and ask him to "sort them by number in descending order of relevance."

Jev: I create a scaled-rate question for each of the 20 chunks, asking "How relevant is this question?", and then I send all 20 questions together in one request.

```
{ "model": "jev-1.13.0", "state": { "query": "What is the deadline for transportation expense reimbursement?", "items": [ { "id": "item_0", "title": "Travel and Transportation Expense Regulations", "text": "Transportation expense reimbursement requests must be submitted by the 25th of each month." }, { "id": "item_1", "title": "Travel and Transportation Expense Regulations", "text": "To request transportation expense reimbursement, enter the date of use, travel route, business destination, and purpose into the expense reimbursement system..." } // ... up to item_19 ] }, "questions": { "rel_0": { "type": "score", "instructions": "How relevant is the text of item_0 to the query?", "criteria": [ "0 - Irrelevant. It has nothing to do with the query.", "1 - Loosely related because it is in the same general area.", "2 - It concerns the same subject, but addresses a different matter from what the query is asking.", "3 - It concerns the same subject and the same matter as the query. It does not matter whether the answer itself is explicitly written." ] } } // ... up to rel_19, one question for each chunk }
```

The graded evaluation **criteria** is an arrangement of descriptions sorted from 0 points upwards.

The description for a 3-point rating includes the phrase "whether or not the answer is written is irrelevant" to prevent the relevance from being influenced by whether or not the answer is present. The results are sorted in descending order of the returned values `score`.

## Results

Press enter or click to view image in full size

| method            | Recall@1 | Recall@5 | nDCG@5 | Speed | 1,000 queries |
|-------------------|----------|----------|--------|-------|---------------|
| No reranking      | 0.800    | 1.000    | 0.814  | —     | —             |
| Cohere Rerank 3.5 | 0.950    | 1.000    | 0.884  | 198ms | \$2.00        |
| Amazon Rerank 1.0 | 0.850    | 0.967    | 0.819  | 337ms | \$1.00        |
| Claude Haiku 4.5  | 1.000    | 1.000    | 0.960  | 981ms | \$2.73        |
| Jev               | 0.983    | 1.000    | 0.965  | 290ms | \$0.25        |

First of all, Haiku is out of the question (it takes almost a second and is the most expensive). Calling an LLM just for sorting and then excluding it doesn't seem worthwhile.

As for the two rerankers and Jev, there wasn't much difference in accuracy or speed. The difference was in price, with Jev being 1/8th. You might think Jev is great!

But this accuracy isn't very reliable. A Recall@5 score of 1.000 without reranking means that the answer was already in the top 5 when the vector search was performed. It seems like there was no need for the reranker at all.

## Verification 2: Can you determine if the answer is "I cannot answer"?

Even if you get the chunks, it's meaningless if the answer isn't there. If you can determine that a near-miss question "cannot be answered," you can respond with "that information could not be found" without calling an LLM, which saves costs.

## How to judge

Rerank score threshold: If the maximum score returned by a reranker is below the threshold, the answer is not allowed. This is a method that has been used in RAG for a long time.

The threshold was determined using the 40 questions before opening the test. Cohere is 0.672, and Amazon is 0.0006. Since the score level differs for each method, it needs to be determined separately.

ClaudeHaiku4.5: You are given a question and the top 5 chunks, and the player answers yes or no to the question "Can you answer with just this information?"

Jev: I'll ask yes/no questions along with the top 5 chunks.

The five results provided are the top five results sorted using CohereRerank 3.5 for all three methods.

```
{ "model": "jev-1.13.0", "state": { "query": "By when do I need to apply for the social insurance premium exemption while on childcare leave?", "items": [ { "id": "item_0", "title": "Childcare and Family Care Leave Regulations", "text": "Social insurance premiums during childcare leave are exempted upon request by the employee. The Human Resources Department handles the procedure." } // ... up to item_4 ] }, "questions": { "can_answer": { "type": "noul", "instructions": "Do the texts in the items contain evidence that can answer the query?", "criteria": { "true": "At least one of the items directly states the value, condition, method, or contact point requested by the question, so the question can be answered using only the items.", "false": "The items discuss the same topic as the question, but do not state the specific information being asked for. Or they only point to another source (for example, 'as separately specified' or 'check with the responsible department'). Or they concern a different subject or matter. Answering the question would require information that is not provided." } } } }
```

## Results

Press enter or click to view image in full size

| method                         | Accur<br>acy | Near<br>miss rate | Possible false<br>blocking rate | Additional time | additional cost |
|--------------------------------|--------------|-------------------|---------------------------------|-----------------|-----------------|
| Cohere Rerank<br>3.5 threshold | 65.0%        | 25.0%             | 53.3%                           | +0ms            | +\$0            |
| Amazon Rerank<br>1.0 threshold | 65.8%        | 95.0%             | 3.3%                            | +0ms            | +\$0            |
| Claude Haiku 4.5               | 98.3%        | 2.5%              | 1.7%                            | +584ms          | +\$1.04         |
| Jev                            | 99.2%        | 0.0%              | 1.7%                            | +263ms          | +\$0.05         |

### Verification 3: Can we do reranking and answer judgment together?

At this point, I'm getting ambitious. Since Jev can process questions in parallel, maybe I can do the reranking and answer determination in a single request? So, I'll send two questions simultaneously for each chunk.

```
{ "model": "jev-1.13.0", "state": { "query": "By when do I need to apply for the social insurance premium exemption while on childcare leave?", "items": [ { "id": "item_0", "title": "Childcare and Family Care Leave Regulations", "text": "Social insurance premiums during childcare leave are exempted upon request by the employee. The Human Resources Department handles the procedure." } // ... up to item_19 ] }, "questions": { "rel_0": { "type": "score", "instructions": "How relevant is the text of item_0 to the query?", "criteria": [ "0 - Irrelevant. It has nothing to do with the query.", "1 - Loosely related because it is in the same general area.", "2 - It concerns the same subject, but addresses a different matter.", "3 - It concerns the same subject and the same matter as the query." ] }, "ans_0": { "type": "noul", "instructions": "Does the text of item_0 contain evidence that can answer the query?", "criteria": { "true": "The item's text directly states the rule, condition, value, procedure, or person responsible that answers the question.", "false": "The item's text discusses the same topic but does not state the rule, condition, value, procedure, or person responsible that the question asks for. Or it concerns a different topic." } } // ... up to item_19, with 2 questions per chunk (40 questions total) } }
```

This allows us to sort by relevance and `noul` filter the results so that if the highest-relevance chunk doesn't reach 0.755, it's not answerable. Only yes/no questions have been added; the number of requests remains at one.

We compared three different configurations.

| composition | nDCG | Accuracy of     | Spe | Cost per      |
|-------------|------|-----------------|-----|---------------|
|             | 0.5  | answer judgment | ed  | 1,000 queries |